Precept 1

A Brief Introduction to R

Introduction

(Please present how to download R&Rstudio on your computer)

How to download R and R studio? Download R: Many mirrors can be used to download R on this website https://cran.r-project.org/mirrors.html

R studio: Free Rstudio Desktop is enough
https://www.rstudio.com/products/rstudio/download/

Note:Please ***make sure you have already installed an R*** before installing Rstudio.

Here are some very basic things you can do in R:

Calculator

R can be used as a calculator: any expressions that you type into the R console will be computed, and the answer printed. For example

```
1 + 1
```

```
## [1] 2
```

```
5 / 9
```

```
## [1] 0.5555556
```

Comments

You can write comments in R using the hash symbol #.

```
# This line of code will not get evaluated.  2 + 2
## The line below this one will be though
2 + 2
```

```
## [1] 4
```

Vectors

You can form vectors in R using the command `c`. Operations are

usually applied element-byelement to a vector.

```
# Creating a vector

c(1, 2, 3)
```

```
## [1] 1 2 3
```

```
# Adding two vectors together

c(1, 2, 3) + c(4, 5, 6)
```

```
## [1] 5 7 9
```

```
# Multiplying two vectors together

c(1, 2, 3) * c(1, 2, 3) #elementwise,
not dot product
```

```
## [1] 1 4 9
```

Matrix

```
X <- matrix(nrow=2,ncol =2)
X[1 ,1] <- 1
X[1 ,2] <- 0
X[2 ,1] <- 0
X[2 ,2] <- 1
print(X)
```

```
##      [,1] [,2]
## [1,]    1    0
## [2,]    0    1
```

```
# Matrix Operation
```

```
Y <- matrix(c(0,2,1,0),nrow=2,byrow=TRUE)
  print(X+Y)
```

```
##      [,1] [,2]
## [1,]    1    2
```

```
## [2,]    1    1
```

```
print(X*Y)
```

```
##      [,1] [,2]
## [1,]    0    0
## [2,]    0    0
```

```
print(X%*%Y) #inner  product
```

```
##      [,1] [,2]
## [1,]    0    2
## [2,]    1    0
```

Dataframe

```
kids  <- c("Jack","Jill") ages  <- c(12 ,10) d <-
data.frame(kids ,ages ,stringsAsFactors=FALSE)
print(d)
```

```
##   kids ages
## 1 Jack   12
## 2 Jill   10
```

Naming Objects

You can “save” objects in memory by using either the arrow <- or an

equal sign =. We call this action "assignment to a name".

```
# Assigning the vector (1, 2, 3) to x
x <- c(1, 2, 3)
# Assigning the vector (4, 5, 6) to y
y <- c(4, 5, 6)
# Adding x and y should return (5, 7, 9)

x + y
```

```
## [1] 5 7 9
```

There are some rules for naming objects in R:

- Names cannot start with a number, so, for example `001graph` would not be valid.
- Names cannot contain punctuation, so `n*mmatrix` is not legitimate. The only two exceptions are periods `.` and underscores `_`.
- Names are case-sensitive, so `vorpal1` and `Vorpal1` are two different objects

Functions

All functions in R are called using their names followed by their arguments in parentheses (more on this later). For example, here we take the average of `x`, which was assigned the values 1, 2, and 3 earlier.

```
mean(x)
```

```
## [1] 2
```

Getting Help in R

To get help about a function in R, you can use `help(function_name)` or `?function_name`. For example, the following calls would fetch information about the mean function.

```
help(mean) ?mean
```

More often than not, though, I end up deferring to StackExchange or Google.

Some useful statistics function:

(Introduce the definitions of these statistics first)

```
x <- c(1,1,2,3,5,7,9)

mean(x) ## [1] 4

median(x) ## [1] 3

sd(x) ## [1] 3.109126

min(x) ## [1] 1

max(x) ## [1] 9

range(x) ## [1] 1,9
```

Build a function

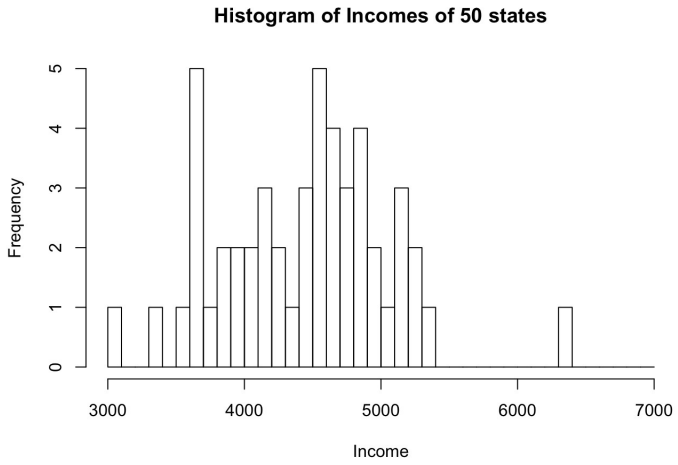
```
add <-
function(x,y){   c=x+y
  return(c)
}
#you must write the value that you want to return


a <- 3


b <- 4


result <- add(a,b)
```

```
print(result)
```

```
## [1] 7
```

Draw histogram

```
state =    data.frame(state.x77) #Data   sets   related
to the 50  states  of the  Unite States
Income=state$Income
hist(Income ,xlab="Income",ylab="Frequency",breaks=seq
(3000,7000,100)) #explain how to choose breaks
```



Logical Statements

R understands the following logical checks:

`==, >=, <=, <, >, !=`

The == checks for equality. Note that it's two equal signs! This is a common mistake.

```
5 == 2+3
```

```
## [1] TRUE
```

```
c(1,2) == c(1,5)
```

```
## [1]  TRUE FALSE
```

The `!=` checks for nonequality. Usually the `!` in programming denotes negation.

```
4 != 2+3
```

```
## [1] TRUE
```

Another thing to note is that R performs these checks element-byelement for vectors.

```
c(1, 2, 3, 4, 5) <= 3
```

```
## [1]  TRUE  TRUE  TRUE FALSE FALSE
```

Subsetting in R

You can pick out certain elements of a vector using square brackets

```
x <- c(1, 3, 5, 7, 9)

x[2] ## [1] 3

x[c(1, 3, 5)] ## [1]

1 5 9

x[c(TRUE, FALSE, FALSE, FALSE, TRUE)]

## [1] 1 9
```

Special Values

There are some special values:

- NULL is completely empty, no data type, no length
- NA denotes a missing value. It takes on a data type!
- Inf is positive infinity
- NaN is Not a Number